

GMSK调制解调 Matlab仿真

GMSK介绍

1979年由日本国际电报电话公司提出的GMSK调制方式。有较好的功率频谱特性，较低的误码性能，特别是带外辐射小，很适用于工作在VHF和UHF频段的移动通信系统，越来越引起人们的关注。GMSK调制方式的理论研究已较成熟。实际应用却还不多，主要是由于高斯滤波器的设计和制作在工程上还有一定的困难。高斯最小频移键控(GMSK)由于带外辐射低因而具有很好的频谱利用率，其恒包络的特性使得其能够使用功率效率高的C类放大器。这些优良的特性使其作为一种高效的数字调制方案被广泛的运用于多种通信系统和标准之中。

其中包括:

- 依据欧洲通信标准化委员会(ETSI)制定的GSM技术规范研制而成的全球通(GSM)数字蜂窝移动系统;
- 由欧洲邮政与电信协会(CEPT)制定的作为欧洲通信标准ETS1300—175的无绳通信标准(DECT);
- 英国和香港，基于无绳电话(CordlessPhones)和电信点(Telepoint)系统的通信标准，CT-2和CT-3系统;
- 基于爱立信公司提出的Mobitex协议的，Mobitex系统(欧洲)和RAM移动数据系统(美国);
- 建立在北美高级移动电话系统(AMPS)上实现无线数据业务的蜂窝数字分组数据(CDPD)系统;
- 第三代个人通信系统(PCS)中，美国的基于GSM标准的PCS1900;以及欧洲的由ETIS开发和制定的个人通信网(PCN)标准DCSI 800;
- 作为欧洲无线局域网(WLAN)标准的HiperLAN /1以及如今讨论的很多的作为无线个人网络(WPAN)标准的蓝牙(Bluetooth)系统;
- 专用系统中有根据国际民航组织(ICAO)制定的卫星通信、导航、搜索/空中交通管理} CNS /ATM)系统等;
- 通用分组无线服务(GPRS)以及改进数据率GSM服务(EDGE)作为由第二代通信标准向第三代通信标准过渡方案也是以GMSK作为其调制方案;

1999年，国际电联ITU着手建立的第三代无线通信标准IMT2000体系。根据不同的应用和技术将其分成5大类:

- (1) IMT—DS: 基于ETSI的W-CDMA技术，采用直序列扩频技术的CDMA方案;
- (2) IMT—MC: 基于北美的CDMA One，采用多载波CDMA技术;
- (3) IMT—TC: 基于ETSI的TD-CDMA技术，采用时分双工(TDD)和TDMA/CDMA的多址方式;
- (4) IMT—SC: 基于UWC—136/EDGE网络;
- (5) IMT—FT: 基于采用FDM.4的DECT技术。

其中后三类无线接口的调制方式都采用GMSK技术或者与之兼容。

GMSK有着广泛的应用。因此，从上世纪80年代提出该技术以来，广大科研人员进行了大量的针对其调制解调方案的研究。

GMSK的原理分析

GMSK的基本原理

高斯滤波最小频移键控（Gaussian Filtered Minimum Shift Keying, GMSK）是一种在MSK基础上改进而来的数字调制方式。其基本原理是将基带信号先通过高斯低通滤波器进行脉冲成形，再进行MSK调制，如图1所示。

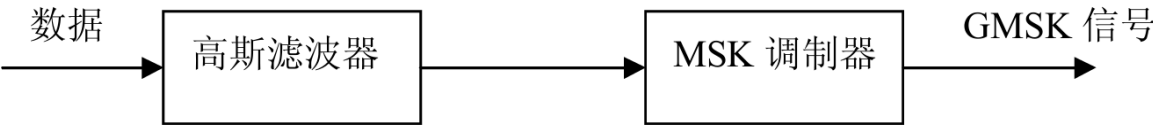


图1 GMSK信号调制基本原理图

该高斯滤波器能够将基带信号变换为高斯脉冲信号，其包络无陡峭边沿和拐点，从而改善MSK信号的频谱特性。高斯滤波器平滑了MSK信号的相位曲线，稳定了信号的频率变化，显著降低了发射频谱的旁瓣电平。

为实现GMSK调制，需设计一个性能良好的高斯低通滤波器，其应具备以下特性：

- 良好的窄带特性和尖锐的截止特性，以滤除基带信号中的高频成分；
- 脉冲响应过冲量小，防止已调波瞬时频偏过大；
- 输出脉冲响应曲线面积对应的相位为 $\frac{\pi}{2}$ ，使调制指数为 0.5。

高斯低通滤波器的脉冲响应 $h(t)$ 可表示为：

$$h(t) = \sqrt{\frac{2\pi}{\ln 2}} B_{3\text{dB}} \exp \left\{ -\frac{2}{\ln 2} (\pi B_{3\text{dB}} t)^2 \right\}$$

其中， $B_{3\text{dB}}$ 为高斯滤波器的3dB带宽。

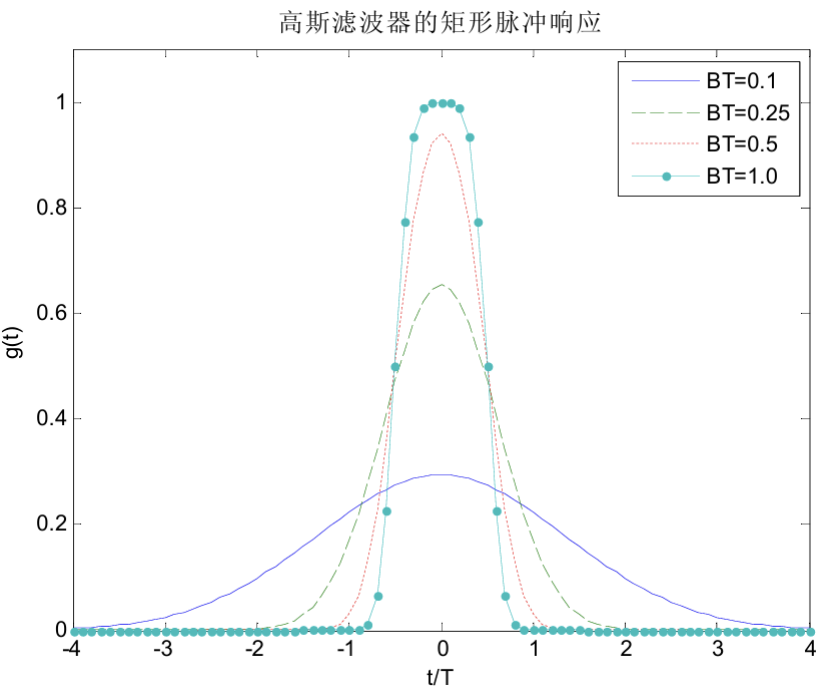


图2 高斯滤波器的矩形脉冲响应

其矩形脉冲响应 $g(t)$ 为：

$$g(t) = h(t) * \text{rect}\left(\frac{t}{T}\right)$$

其中, $\text{rect}(x)$ 为矩形函数, 定义为:

$$\text{rect}(x) = \begin{cases} 1, & |x| < \frac{1}{2} \\ 0, & \text{otherwise} \end{cases}$$

经推导, $g(t)$ 可表示为:

$$g(t) = \frac{1}{2} \left[\text{erf} \left(-\sqrt{\frac{2}{\ln 2}} \pi B_{3\text{dB}} \left(t - \frac{T}{2} \right) \right) + \text{erf} \left(\sqrt{\frac{2}{\ln 2}} \pi B_{3\text{dB}} \left(t + \frac{T}{2} \right) \right) \right]$$

其中, $\text{erf}(x)$ 为误差函数:

$$\text{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt$$

已调信号的相位表达式为:

$$\theta(t) = \frac{\pi}{2T} \int_{-\infty}^t \left[\sum_n a_n g \left(\tau - nT - \frac{T}{2} \right) \right] d\tau$$

其中, $a_n \in \{\pm 1\}$ 为NRZ码元, 调制指数 $h = 0.5$, 保证一个码元时间内最大相位变化为 π 。

最终, GMSK信号表达式为:

$$s(t) = \cos \left\{ \omega_c t + \frac{\pi}{2T} \int_{-\infty}^t \left[\sum_n a_n g \left(\tau - nT - \frac{T}{2} \right) \right] d\tau \right\}$$

由于高斯滤波器引入了码间干扰 (即部分响应波形), 使得相邻码元之间相互影响。该影响程度与高斯滤波器的3dB带宽 B 和码元宽度 T 的乘积 BT 有关。 BT 值越小, 频谱越紧凑, 带外辐射越小, 但码间干扰越大。

高斯滤波器的输出脉冲经MSK调制得到GMSK信号, 其相位路径由脉冲的形状决定。由于高斯滤波后的脉冲无陡峭沿, 也无拐点, 因此, 相位路径得到进一步平滑。

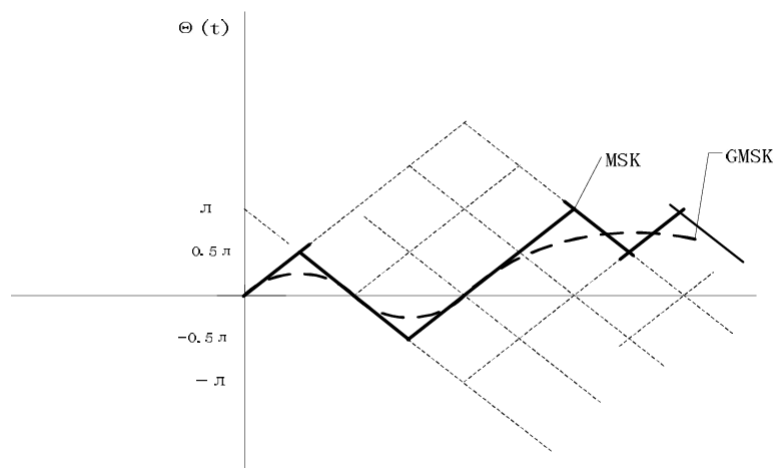


图3 GMSK相位路径图****

GMSK信号的调制与解调

GMSK信号的调制原理

直接数字调频方案：该方案利用脉冲形成后的基带信号直接对压控振荡器VCO进行调频。该方案十分简单，并且在多种模拟和数字系统中采用。例如蜂窝数字分组数据系统(CDPD)和全球通(GSM)。可是该方案不易于集成。而且为了保持中心频率在动态范围内，就必须要求VCO有着较高的线性度和灵敏度。类似的方案还有环路型调制器(见图4)。2 BPSK保证每个码元得相位变化为2，利用锁相环对相位进行平滑。可是如何设计PLL的传输函数，从而满足功率谱特性的需要是一件很困难的事情。

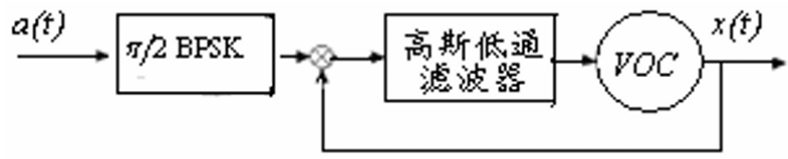


图4 锁相环型GMSK调制器

正交平衡式调制器 :我们可以将GMSK信号写作:

$$\begin{aligned} s(t) &= \cos(2\pi f_c t + \theta(t)) \\ &= \cos \theta(t) \cos(2\pi f_c t) - \sin \theta(t) \sin(2\pi f_c t) \end{aligned}$$

其中, $\theta(t) = \frac{\pi}{2T} \int_{-\infty}^t \left[\sum a_n g \left(\tau - nT - \frac{T}{2} \right) \right] d\tau$

因此,同MSK信号类似,GMSK信号也可以采用正交平衡式调制器。其原理框图如图5

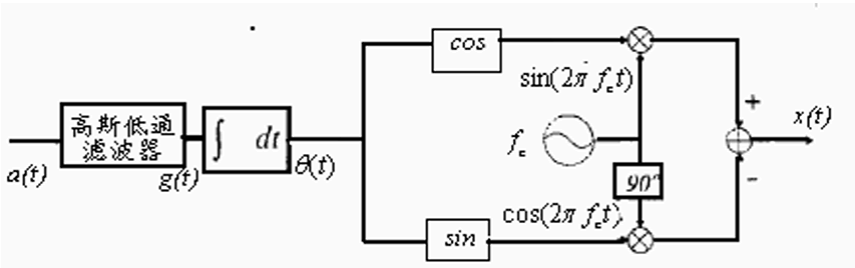


图5 正交平衡调制器

该方案有着实现简单,容易实现数字化,以达到最终的大规模集成。但是其缺点是,同相和正交两路输出信号必须进行平衡,否则会出现附加调幅,导致频带的扩展。如果采用利用RAM储存波形的方案,调制时直接查表取值则不用担心平衡问题。可是,当载波频率很高时(例如GSM中的载波频率为900MHz),D/A转换器,以及DSP处理器的速度则成为制约该方案的主要因素。这时候,就需要通过调整同相、正交电路来达到平衡的目的。

GMSK的解调原理

GMSK 信号的相干解调 :相干解调技术在基于GMSK调制体制的系统中应用十分广泛。例如,GSM系统中在其基站部分和移动端部分都使用相干解调技术。如果使用相干解调技术,接收机需要知道参考相位,或者进行精确的载波恢复。这也要求接收机拥有本振、锁相环路、以及载波恢复电路等部分,这些都使得接收机的复杂程度和成本增加。

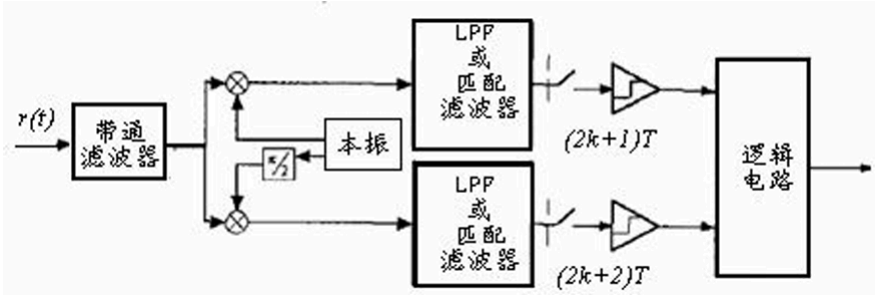


图6 相干解调器

GMSK信号可以类似的采用MSK正交平衡调制方案。因此我们可以并行的实现对它的解调。而且,还可以通过分别对同相部分和正交部分进行相干解调来达到性能的优化。

调制器和解调器的两个相互正交的通道必须进行时钟同步、幅值平衡、以及相位正交,否则系统的性能就会降低。可是随着数据传输速率的提高,其实现的难度也增加了。MSK的串行实现方案避免了并行方案中平衡以及时域同步的问题。但是,它对带通滤波器以及匹配滤波器的精度要求很高。我们可以将GMSK视为QPSK的一种特殊的形式。也就是将码元序列的奇、偶位码元分别调制在同相载波和正交载波上。分别对于这两个正交的载波部分来说,其信息的传输速率为 $1/2T$ (bps)。而总的传输速率为 $1/T$ (bps)。接收端奇偶码元的判决时间为 $2T$ 。文献[10]指出在高斯加性白噪声的情况下和基于正交GFSK使用匹配滤波器相干解调方案比较,基于GMSK的并行相干解调方案可以有大约3dB(E_b/N_0)增益。

系统为补偿由于多径传播产生的时延扩展以及预调制滤波器和检测前滤波器引入的码间干扰,往往在相干解调方案中采用线性均衡技术。可是,由于对接收信号相位的跟踪不良而造成的误码仍然无法消除。因此,在衰落环境下的相干解调技术的性能并不十分理想。

GMSK信号的非相干解调:对于GMSK信号可以采用多种非相干技术进行解调。非相干解调技术不需要知道参考相位,因此也就不需要锁相环路、本地晶振以及载波恢复电路了。相对与相干解调技术,非相干解调技术的成本更低,更易于实现。非相干解调技术的种类很多。主要分为限幅鉴频器和差分解调两个大类,以及基于这两大类技术的多种衍生方案。本节分别给出了两种方案的原理图,并对目前关于该方向的研究状况和主要成果进行综述。

- 限幅鉴频器解调:

限幅鉴频器,顾名思义由两个部分组成:限幅器,用来恢复受到噪声和干扰影响的接收信号的恒包络的特性;鉴频器,用来将相位调制转化为幅度调制,以供随后的包络检测。鉴频器之后通常跟随一个低通滤波器,例如一个积分滤波器。信号通过低通滤波器之后进入判决器判决,如图7

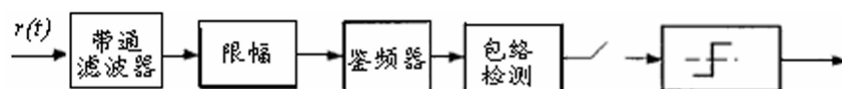


图7 限幅解调器

- 非相干差分解调:

非相干差分解调,利用接收信号以及其时延信号进行解调。原理图如图8所示:其中C代表一个复常数(当延时为T时, $C=-j$)。

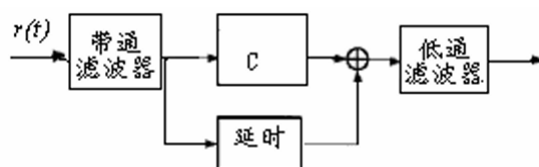


图8 非相干差分解调器

相干与非相干解调方案各有利弊。相干解调在AWGN环境下,其BER指标更好。但是,在衰落环境下,由于受到载波恢复电路中的锁相环性能的限制,使相干解调的性能下降,误码率增加。而且随机的调频噪声,将部分地影响相邻符号时间内的载波相位的相关性。而对于差分解调来说,则不存在这些因素。因此我们说,在快衰落环境下,非相干解调的性能更优。

关于相干解调技术的研究已经非常成熟了。相对来说,非相干技术—差分解调的研究较少。而且如前所述差分解调在衰落信道中有着较好的性质。且常规的差分解调相对于相干解调在AWGN环境下有着大约8dB的差距(BER),这同时也意味着还有很大的性能提高的空间。

接收端模块（解调部分）

1.I/Q信号分离

从接收信号中提取同相和正交分量,为后续相关检测做准备

2.相关检测器

```
1 % 相关运算
2 result1 = dot(Sn1, Sn) + dot(Cn1, Cn);
3 result2 = dot(Sn2, Sn) + dot(Cn2, Cn);
4 计算接收信号与候选信号的相关值
```

3.状态转移表

```
1 TransferTable = [
2     0 +1 +1 pi/2 +1 -1 pi -1 +1 pi/2 +1 +1
3     0 +1 +1 pi/2 +1 -1 pi -1 +1 pi/2 +1 -1
4     ... % 8条完整路径
5 ];
```

4.维特比算法处理

```
1 % 路径选择逻辑
2 if result1 >= result2
3     temp_path = [temp_path; TransferPath(idx, 4:6) TransferPath(idx,1:3)];
4     temp_hamming = [temp_hamming; result1];
5 else
6     temp_path = [temp_path; TransferPath(idx, 7:9) TransferPath(idx,1:3)];
7     temp_hamming = [temp_hamming; result2];
8 end
```

实现最大似然序列检测,处理GMSK固有的码间干扰

5.回溯解码

从最终路径度量回溯找到最优路径,输出解码后的比特序列

仿真效果

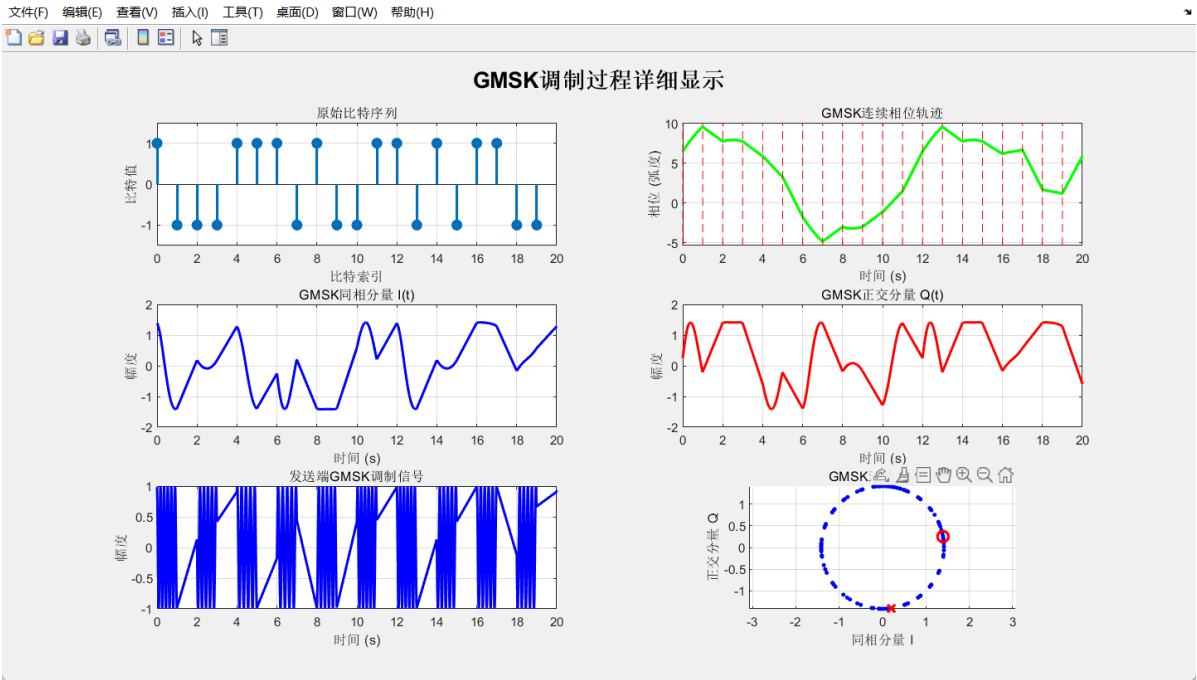


图1 GMSK调制过程详细显示

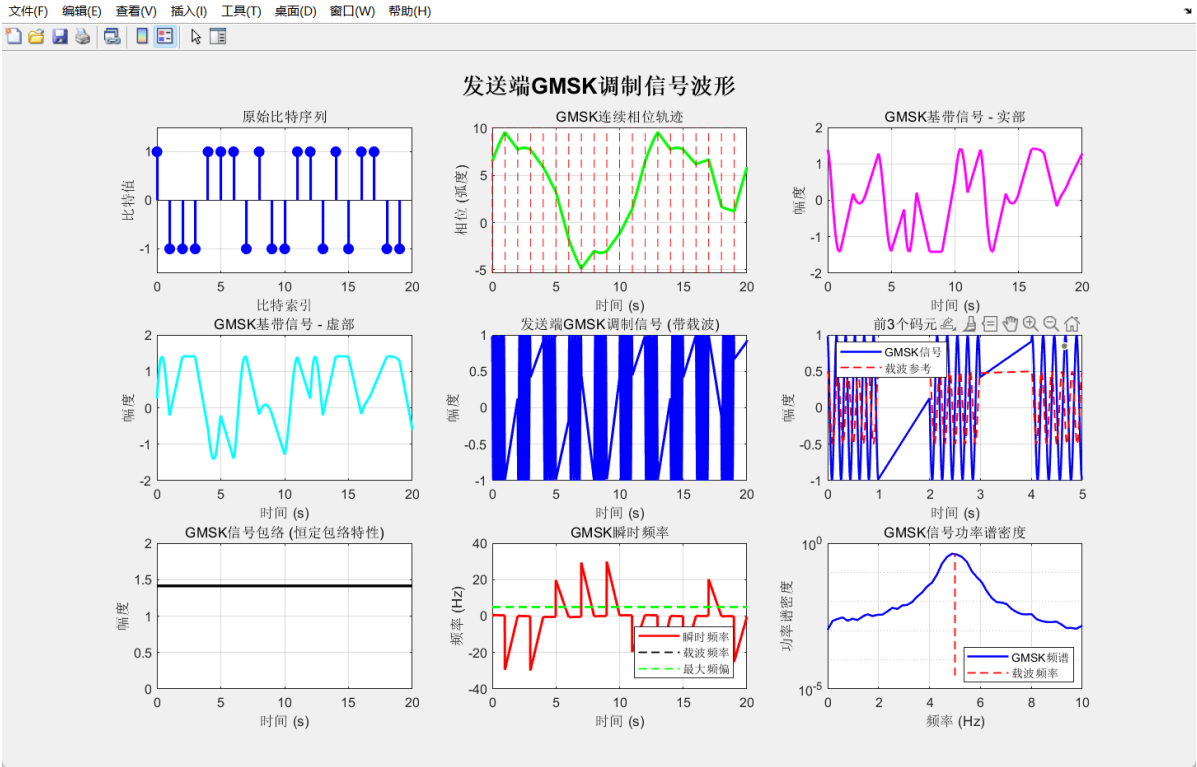


图2 发送端GMSK调制信号波形

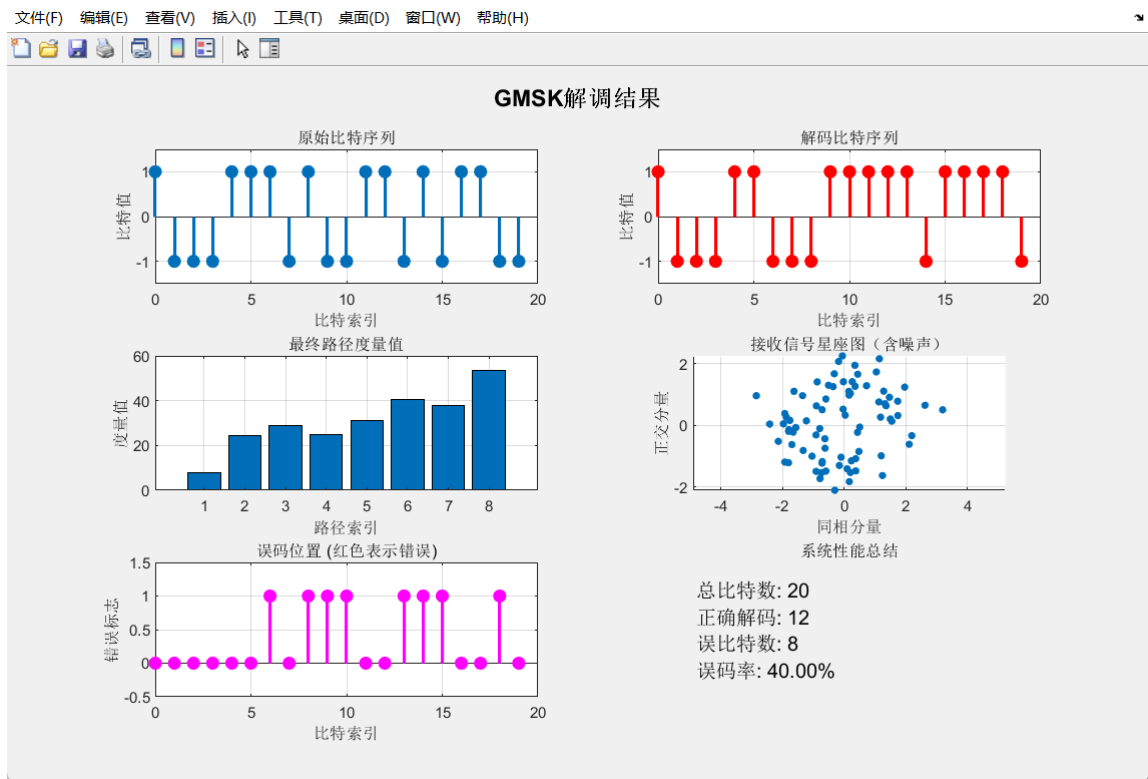


图3 GMSK解调结果

参考文献

- [1]王士林, 陆存乐, 龚初光. 现代数字调制技术[M]. 人民邮电出版社, 1985.
- [2]张辉, 曹丽娜. 现代通信原理与技术.西安: 西安电子科技大学出版社, 2002.
- [3]曹志刚, 钱亚生. 现代通信原理. 北京: 清华大学出版社, 2002.
- [4] Hiroshi Harada, Ramjee Presad, Simulation and Software Radio for Mobile Communications
- [5]杨小牛. 软件无线电原理与应用[M]. 北京: 电子工业出版社, 2001
- [6]杨允均, 武传华. “用MATLAB实现GMSK信号产生与解调”, 工程应用, 2005
- [7]郭梯云. 移动通信原理. 北京: 人民邮电出版社, 2000
- [8] William H. Tranter, etc 通信系统仿真原理与无线应用, 机械工业出版社, 2005
- [9] 楼顺天, 姚若玉, 沈俊霞. MATLAB7.0 程序设计语言, 西安: 西安电子科技大学出版社, 2007.
- [10] Dornstetter J.L., Verhulst D. Cellular efficiency with slow frequency hopping: analysis of the digital SFH900 mobile system. IEEE Journal on Selected Areas in Commun, 1987.

附录:

```
1 %% GMSK调制解调系统完整仿真
2 clear; clc; close all;
3
4 %% 参数设置
5 N_bits = 20; % 比特数
6 Tb = 1; % 码元周期
7 dt = 0.01; % 采样间隔
8 t = 0:dt:N_bits*Tb-dt; % 时间轴
9 E = 1; % 信号能量
```

```

10 fc = 5; % 载波频率 (Hz)
11
12 %% 生成随机比特序列
13 original_bits = [1, -1, -1, -1, 1, 1, 1, -1, 1, -1, -1, 1, 1, -1, 1, -1, 1,
14 1, -1, -1];
15
16 fprintf('原始比特序列: ');
17 disp(original_bits);
18
19 %% GMSK调制过程
20 fprintf('\n=== GMSK调制过程 ===\n');
21
22 % 高斯脉冲成形函数
23 gt = @(t) (erfc(2*pi*0.3*(t - 2) / sqrt(2*log(2))) - erfc(2*pi*0.3*(t + 1)
24 / sqrt(2*log(2)))) / 2;
25
26 % 初始化相位和信号
27 theta = [0, 1, 1]; % 初始相位状态
28 modulated_I = []; % 同相分量
29 modulated_Q = []; % 正交分量
30 phase_history = []; % 相位历史
31 time_history = []; % 时间历史
32 gmsk_signal = []; % GMSK调制信号
33 gmsk_transmit = []; % 发送端GMSK信号（带载波）
34
35 % 调制每个比特 - 高采样率用于显示连续波形
36 for n = 1:N_bits
37     % 计算当前比特的相位贡献
38     current_bit = original_bits(n);
39
40     % 更新相位状态
41     next_theta = [theta(1) + theta(2) * pi/2, theta(3), current_bit];
42
43     % 生成高采样率的调制信号用于显示连续波形
44     [Sn_high, Cn_high, phase_high, t_high, gmsk_high, gmsk_transmit_high] =
45     ...
46     gmsk_modulate_high_res([next_theta, theta], n-1, gt, Tb, dt, E,
47     fc);
48
49     % 生成用于解调的4点采样信号
50     [Sn, Cn, phase] = gmsk_modulate([next_theta, theta], n-1, gt);
51
52     % 添加噪声到解调信号
53     Sn_noisy = Sn + 0.5*randn(1,4);
54     Cn_noisy = Cn + 0.5*randn(1,4);
55
56     % 存储信号
57     modulated_I = [modulated_I, Sn_noisy];
58     modulated_Q = [modulated_Q, Cn_noisy];
59     phase_history = [phase_history, phase_high];
60     time_history = [time_history, t_high + (n-1)*Tb];
61     gmsk_signal = [gmsk_signal, gmsk_high];
62     gmsk_transmit = [gmsk_transmit, gmsk_transmit_high];
63
64     % 更新相位状态

```

```

62     theta = next_theta;
63 end
64
65 %% 绘制发送端GMSK调制波形 - 专门窗口
66 figure('Position', [50, 50, 1400, 1000]);
67 sgtitle('发送端GMSK调制信号波形', 'FontSize', 16, 'Fontweight', 'bold');
68
69 % 1. 原始比特序列
70 subplot(3,3,1);
71 bit_time = 0:N_bits-1;
72 stem(bit_time, original_bits, 'filled', 'b', 'Linewidth', 2);
73 title('原始比特序列');
74 xlabel('比特索引');
75 ylabel('比特值');
76 grid on;
77 ylim([-1.5, 1.5]);
78
79 % 2. 连续相位轨迹
80 subplot(3,3,2);
81 plot(time_history, phase_history, 'g-', 'Linewidth', 2);
82 hold on;
83 % 标记每个比特开始的位置
84 for n = 1:N_bits
85     plot([(n-1)*Tb, (n-1)*Tb], [min(phase_history)-0.5,
86 max(phase_history)+0.5], 'r--', 'Linewidth', 0.5);
87 end
88 title('GMSK连续相位轨迹');
89 xlabel('时间 (s)');
90 ylabel('相位 (弧度)');
91 grid on;
92 xlim([0, N_bits*Tb]);
93
94 % 3. GMSK基带信号 - 实部
95 subplot(3,3,3);
96 plot(time_history, real(gmsk_signal), 'm-', 'Linewidth', 1.5);
97 title('GMSK基带信号 - 实部');
98 xlabel('时间 (s)');
99 ylabel('幅度');
100 grid on;
101 xlim([0, N_bits*Tb]);
102
103 % 4. GMSK基带信号 - 虚部
104 subplot(3,3,4);
105 plot(time_history, imag(gmsk_signal), 'c-', 'Linewidth', 1.5);
106 title('GMSK基带信号 - 虚部');
107 xlabel('时间 (s)');
108 ylabel('幅度');
109 grid on;
110 xlim([0, N_bits*Tb]);
111
112 % 5. 发送端GMSK信号 - 完整波形
113 subplot(3,3,5);
114 plot(time_history, gmsk_transmit, 'b-', 'Linewidth', 1.5);
115 title('发送端GMSK调制信号 (带载波)');
116 xlabel('时间 (s)');
117 ylabel('幅度');

```

```

117 grid on;
118 xlim([0, N_bits*Tb]);
119
120 % 6. 发送端GMSK信号 - 前3个码元细节
121 subplot(3,3,6);
122 end_idx = min(length(time_history), round(3*Tb/dt));
123 plot(time_history(1:end_idx), gmsk_transmit(1:end_idx), 'b-', 'Linewidth',
124      1.5);
125 hold on;
126 % 叠加载波参考
127 carrier_ref = cos(2*pi*fc*time_history(1:end_idx));
128 plot(time_history(1:end_idx), 0.5*carrier_ref, 'r--', 'Linewidth', 1);
129 title('前3个码元GMSK信号细节');
130 xlabel('时间 (s)');
131 ylabel('幅度');
132 legend('GMSK信号', '载波参考', 'Location', 'best');
133 grid on;
134
135 % 7. GMSK信号包络
136 subplot(3,3,7);
137 envelope = abs(gmsk_signal);
138 plot(time_history, envelope, 'k-', 'Linewidth', 2);
139 title('GMSK信号包络 (恒定包络特性)');
140 xlabel('时间 (s)');
141 ylabel('幅度');
142 grid on;
143 xlim([0, N_bits*Tb]);
144 ylim([0, 2]);
145
146 % 8. 瞬时频率变化
147 subplot(3,3,8);
148 % 计算瞬时频率 (相位差分)
149 instant_phase = unwrap(phase_history);
150 instant_freq = diff(instant_phase) / (2*pi*dt);
151 plot(time_history(1:end-1), instant_freq, 'r-', 'Linewidth', 1.5);
152 hold on;
153 % 标记理论频偏
154 plot([0, N_bits*Tb], [fc, fc], 'k--', 'Linewidth', 1);
155 plot([0, N_bits*Tb], [fc+0.25/Tb, fc+0.25/Tb], 'g--', 'Linewidth', 1);
156 plot([0, N_bits*Tb], [fc-0.25/Tb, fc-0.25/Tb], 'g--', 'Linewidth', 1);
157 title('GMSK瞬时频率');
158 xlabel('时间 (s)');
159 ylabel('频率 (Hz)');
160 legend('瞬时频率', '载波频率', '最大频偏', 'Location', 'best');
161 grid on;
162 xlim([0, N_bits*Tb]);
163
164 % 9. 频谱分析
165 subplot(3,3,9);
166 [Pxx, F] = pwelch(gmsk_transmit, [], [], [], 1/dt);
167 semilogy(F, Pxx, 'b-', 'Linewidth', 1.5);
168 hold on;
169 % 标记载波频率
170 plot([fc, fc], [min(Pxx), max(Pxx)], 'r--', 'Linewidth', 1);
171 title('GMSK信号功率谱密度');
172 xlabel('频率 (Hz)');

```

```

172 ylabel('功率谱密度');
173 legend('GMSK频谱', '载波频率', 'Location', 'best');
174 grid on;
175 xlim([0, 2*fc]);
176
177 %% 绘制调制过程波形 - 单独窗口
178 figure('Position', [200, 200, 1400, 1000]);
179 sgtitle('GMSK调制过程详细显示', 'FontSize', 16, 'FontWeight', 'bold');
180
181 % 1. 原始比特序列
182 subplot(3,2,1);
183 stem(0:N_bits-1, original_bits, 'filled', 'Linewidth', 2);
184 title('原始比特序列');
185 xlabel('比特索引');
186 ylabel('比特值');
187 grid on;
188 ylim([-1.5, 1.5]);
189
190 % 2. 连续相位轨迹
191 subplot(3,2,2);
192 plot(time_history, phase_history, 'g-', 'Linewidth', 2);
193 hold on;
194 % 标记每个比特开始的位置
195 for n = 1:N_bits
196     plot([(n-1)*Tb, (n-1)*Tb], [min(phase_history)-0.5,
197 max(phase_history)+0.5], 'r--', 'Linewidth', 0.5);
197 end
198 title('GMSK连续相位轨迹');
199 xlabel('时间 (s)');
200 ylabel('相位 (弧度)');
201 grid on;
202 xlim([0, N_bits*Tb]);
203
204 % 3. 同相分量 (I)
205 subplot(3,2,3);
206 plot(time_history, sqrt(2*E/Tb)*cos(phase_history), 'b-', 'Linewidth',
207 1.5);
208 title('GMSK同相分量 I(t)');
209 xlabel('时间 (s)');
210 ylabel('幅度');
211 grid on;
212 xlim([0, N_bits*Tb]);
213
214 % 4. 正交分量 (Q)
215 subplot(3,2,4);
216 plot(time_history, sqrt(2*E/Tb)*sin(phase_history), 'r-', 'Linewidth',
217 1.5);
218 title('GMSK正交分量 Q(t)');
219 xlabel('时间 (s)');
220 ylabel('幅度');
221 grid on;
222 xlim([0, N_bits*Tb]);
223
224 % 5. 发送端GMSK信号
225 subplot(3,2,5);
226 plot(time_history, gmsk_transmit, 'b-', 'Linewidth', 1.5);

```

```

225 title('发送端GMSK调制信号');
226 xlabel('时间 (s)');
227 ylabel('幅度');
228 grid on;
229 xlim([0, N_bits*Tb]);
230
231 % 6. 星座图
232 subplot(3,2,6);
233 I_continuous = sqrt(2*E/Tb)*cos(phase_history);
234 Q_continuous = sqrt(2*E/Tb)*sin(phase_history);
235 scatter(I_continuous(1:10:end), Q_continuous(1:10:end), 10, 'filled', 'b');
236 hold on;
237 % 标记起点和终点
238 plot(I_continuous(1), Q_continuous(1), 'ro', 'MarkerSize', 8, 'Linewidth',
239      2);
240 plot(I_continuous(end), Q_continuous(end), 'rx', 'MarkerSize', 8,
241      'Linewidth', 2);
242 title('GMSK连续星座图');
243 xlabel('同相分量 I');
244 ylabel('正交分量 Q');
245 grid on;
246 axis equal;
247
248 %% GMSK解调过程
249 fprintf('\n=== GMSK解调过程 ===\n');
250
251 % 状态转移表
252 TransferTable = [
253     0 +1 +1 pi/2 +1 -1 pi -1 +1 pi/2 +1 +1
254     0 +1 +1 pi/2 +1 -1 pi -1 +1 pi/2 +1 -1
255     0 +1 +1 pi/2 +1 -1 pi -1 -1 pi/2 -1 +1
256     0 +1 +1 pi/2 +1 -1 pi -1 -1 pi/2 -1 -1
257     0 +1 +1 pi/2 +1 +1 pi +1 +1 -pi/2 +1 +1
258     0 +1 +1 pi/2 +1 +1 pi +1 +1 -pi/2 +1 -1
259     0 +1 +1 pi/2 +1 +1 pi +1 -1 -pi/2 -1 +1
260     0 +1 +1 pi/2 +1 +1 pi +1 -1 -pi/2 -1 -1
261 ];
262
263 % 解调路径表
264 TransferPath = [
265     0      -1      +1      pi/2      -1      -1      -pi/2      +1      -1
266     0      +1      +1      -pi/2      +1      +1      pi/2      -1      +1
267     0      +1      -1      -pi/2      +1      +1      pi/2      -1      +1
268     0      -1      -1      pi/2      -1      -1      -pi/2      +1      -1
269     pi     -1      +1      pi/2      +1      -1      -pi/2      -1      -1
270     pi     +1      +1      pi/2      +1      +1      -pi/2      -1      +1
271     pi     +1      -1      pi/2      +1      +1      -pi/2      -1      +1
272     pi     -1      -1      pi/2      +1      -1      -pi/2      -1      -1
273     pi/2    -1      +1      pi      -1      -1      0      +1      -1
274     pi/2    +1      +1      pi     -1      +1      0      +1      +1
275     pi/2    +1      -1      0      +1      +1      pi      -1      +1
276     pi/2    -1      -1      pi      -1      -1      0      +1      -1
277     -pi/2   -1      +1      pi      +1      -1      0      -1      -1
278     -pi/2   +1      +1      0      -1      +1      pi      +1      +1
279     -pi/2   +1      -1      0      -1      +1      pi      +1      +1
280     -pi/2   -1      -1      0      -1      -1      pi      +1      -1

```

```

279 ];
280
281 % 维特比解码初始化
282 theta_now = [0, 1, 1];
283 hamming_head = zeros(8,3);
284 path = [];
285 hamming = [];
286 method = 'a';
287
288 % 解调每个比特
289 for i = 1:N_bits
290     % 提取当前比特的接收信号
291     start_idx = (i-1)*4 + 1;
292     end_idx = i*4;
293     Sn_current = modulated_I(start_idx:end_idx);
294     Cn_current = modulated_Q(start_idx:end_idx);
295
296     % 维特比解码
297     [path_temp, hamming_temp1, hamming_temp2] = viterbi_decode(Sn_current,
298         Cn_current, i-1, method, TransferTable, TransferPath, gt);
299
300     % 路径度量处理
301     if i == 4
302         % 调整前3码元的路径度量
303         hamming_head(5:8,1) = hamming_head(2,1);
304         hamming_head(1:4,1) = hamming_head(1,1);
305         hamming_head(7:8,2) = hamming_head(4,2);
306         hamming_head(5:6,2) = hamming_head(3,2);
307         hamming_head(3:4,2) = hamming_head(2,2);
308         hamming_head(1:2,2) = hamming_head(1,2);
309
310         hamming = zeros(8,1);
311         for k = 1:8
312             hamming(k,1) = sum(hamming_head(k,:));
313         end
314     end
315
316     if i-1 > 3
317         if isempty(hamming)
318             hamming = hamming_temp1;
319         else
320             hamming = [hamming, hamming_temp1];
321         end
322     end
323
324     hamming_head = hamming_head + hamming_temp2;
325
326     % 修复路径存储问题
327     if ~isempty(path_temp)
328         if isempty(path)
329             path = path_temp;
330         else
331             path = [path, path_temp];
332         end
333     end

```

```

334 % 切换方法
335 if i-1 > 3
336     if method == 'a'
337         method = 'b';
338     else
339         method = 'a';
340     end
341 end
342 end
343
344 %% 修复路径回溯部分
345 fprintf('\n=== 最终解码 ===\n');
346
347 % 检查路径矩阵维度
348 if isempty(path)
349     error('路径矩阵为空，无法进行解码');
350 end
351
352 % 路径回溯 - 修复维度问题
353 num_segments = size(path, 2) / 6;
354 if num_segments ~= round(num_segments)
355     error('路径矩阵列数不是6的倍数');
356 end
357
358 realpath = [];
359 new_hamming = [];
360
361 for i = 1:8
362     tempth = path(i, 1:6);
363     temp = path(i, 4:6);
364
365     if size(hamming, 2) >= 2
366         new_hamming_temp = hamming(i, 2);
367     else
368         new_hamming_temp = 0;
369     end
370
371     for k = 1:num_segments - 1
372         start_col = 1 + k*6;
373         end_col = 6 + k*6;
374
375         if end_col > size(path, 2)
376             break;
377         end
378
379         std_segment = path(:, start_col:end_col);
380         found = false;
381
382         for index = 1:8
383             if index <= size(std_segment, 1) && all(temp ==
std_segment(index, 1:3))
384                 tempth = [tempth, std_segment(index, 4:6)]; %#ok<AGROW>
385                 if size(hamming, 2) >= k + 2
386                     new_hamming_temp = [new_hamming_temp, hamming(index, k
+ 2)]; %#ok<AGROW>
387                 end

```



```

388         temp = std_segment(index, 4:6);
389         found = true;
390         break;
391     end
392 end
393
394 if ~found && ~isempty(std_segment)
395     % 如果没有找到匹配, 使用第一个路径
396     tempth = [tempth, std_segment(1, 4:6)]; %#ok<AGROW>
397     if size(hamming, 2) >= k + 2
398         new_hamming_temp = [new_hamming_temp, hamming(1, k + 2)];
399     %#ok<AGROW>
400     end
401     temp = std_segment(1, 4:6);
402 end
403
404 realpath = [realpath; tempth]; %#ok<AGROW>
405 new_hamming = [new_hamming; new_hamming_temp]; %#ok<AGROW>
406 end
407
408 % 最终路径排序
409 realpath2 = [];
410 new_hamming2 = [];
411
412 for i = 1:8
413     tempth = TransferTable(i, 10:12);
414     for k = 1:size(realpath, 1)
415         if k <= size(realpath, 1) && all(tempth == realpath(k, 1:3))
416             realpath2 = [realpath2; realpath(k, :)]; %#ok<AGROW>
417             new_hamming2 = [new_hamming2; new_hamming(k, :)]; %#ok<AGROW>
418             break;
419         end
420     end
421 end
422
423 % 计算最终路径度量
424 if isempty(hamming)
425     finalhamming = sum(hamming_head, 2);
426 else
427     finalhamming = [hamming(:,1), hamming_head, new_hamming2];
428     for i = 1:min(8, size(finalhamming, 1))
429         finalhamming(i,1) = sum(finalhamming(i,2:end));
430     end
431 end
432
433 % 选择最佳路径
434 if ~isempty(finalhamming)
435     [~, best_path_idx] = max(finalhamming(:,1));
436     decoded_path = [TransferTable(:,1:9), realpath2];
437
438     % 提取解码比特
439     decoded_bits = [];
440     for i = 6:3:min(size(decoded_path, 2), 3*N_bits+3)
441         if i <= size(decoded_path, 2)

```

```

442         decoded_bits = [decoded_bits, decoded_path(best_path_idx, i)];
443     %#ok<AGROW>
444     end
445 end
446
447 fprintf('解码比特序列: ');
448 disp(decoded_bits);
449 fprintf('最佳路径号: %d\n', best_path_idx);
450
451 % 计算误码率
452 compare_length = min(length(original_bits), length(decoded_bits));
453 bit_errors = sum(original_bits(1:compare_length) ~=
454 decoded_bits(1:compare_length));
455 ber = bit_errors / compare_length;
456
457 fprintf('误比特数: %d\n', bit_errors);
458 fprintf('误码率: %.4f\n', ber);
459 else
460     decoded_bits = [];
461     bit_errors = N_bits;
462     ber = 1;
463     fprintf('解码失败\n');
464 end
465
466 %% 绘制解调结果
467 figure('Position', [100, 100, 1200, 800]);
468
469 % 原始比特序列
470 subplot(3,2,1);
471 stem(0:N_bits-1, original_bits, 'filled', 'Linewidth', 2);
472 title('原始比特序列');
473 xlabel('比特索引');
474 ylabel('比特值');
475 grid on;
476 ylim([-1.5, 1.5]);
477
478 % 解码比特序列
479 subplot(3,2,2);
480 if ~isempty(decoded_bits)
481     stem(0:length(decoded_bits)-1, decoded_bits, 'filled', 'r',
482 'Linewidth', 2);
483 end
484 title('解码比特序列');
485 xlabel('比特索引');
486 ylabel('比特值');
487 grid on;
488 ylim([-1.5, 1.5]);
489
490 % 路径度量
491 subplot(3,2,3);
492 if ~isempty(finalhamming)
493     bar(finalhamming(:,1));
494 end
495 title('最终路径度量值');
496 xlabel('路径索引');
497 ylabel('度量值');

```

```

495 grid on;
496
497 % 星座图
498 subplot(3,2,4);
499 scatter(modulated_I, modulated_Q, 20, 'filled');
500 title('接收信号星座图 (含噪声)');
501 xlabel('同相分量');
502 ylabel('正交分量');
503 grid on;
504 axis equal;
505
506 % 误码比较
507 subplot(3,2,5);
508 if ~isempty(decoded_bits)
509     compare_length = min(length(original_bits), length(decoded_bits));
510     error_pattern = original_bits(1:compare_length) ~=
511     decoded_bits(1:compare_length);
512     stem(0:compare_length-1, error_pattern, 'filled', 'm', 'Linewidth', 2);
513 end
514 title('误码位置 (红色表示错误)');
515 xlabel('比特索引');
516 ylabel('错误标志');
517 grid on;
518 ylim([-0.5, 1.5]);
519
520 % 系统性能总结
521 subplot(3,2,6);
522 text(0.1, 0.8, sprintf('总比特数: %d', N_bits), 'FontSize', 12);
523 if ~isempty(decoded_bits)
524     text(0.1, 0.6, sprintf('正确解码: %d', N_bits - bit_errors), 'FontSize',
525     12);
526     text(0.1, 0.4, sprintf('误比特数: %d', bit_errors), 'FontSize', 12);
527     text(0.1, 0.2, sprintf('误码率: %.2f%%', ber * 100), 'FontSize', 12);
528 else
529     text(0.1, 0.6, '解码失败', 'FontSize', 12, 'Color', 'red');
530 end
531 axis off;
532 title('系统性能总结');
533
534 sgtitle('GMSK解调结果', 'FontSize', 14, 'Fontweight', 'bold');
535
536 %% 显示系统性能
537 fprintf('\n=== 系统性能总结 ===\n');
538 fprintf('总比特数: %d\n', N_bits);
539 if ~isempty(decoded_bits)
540     fprintf('正确解码比特数: %d\n', N_bits - bit_errors);
541     fprintf('误码率: %.2f%%\n', ber * 100);
542 else
543     fprintf('解码失败\n');
544 end
545
546 %% GMSK调制函数 - 用于解调 (4点采样)
547 function [Sn, Cn, phase] = gmsk_modulate(TransferPath, n, gt)
548     E = 1;
549     Tb = 1;

```

```

549 % 采样时间点
550 t_points = [0.1*Tb, 0.3*Tb, 0.5*Tb, 0.8*Tb];
551 tn_1 = t_points - (n - 1)*Tb;
552 tn_2 = t_points - (n - 2)*Tb;
553 tn = t_points - n*Tb;
554
555 % 计算高斯脉冲响应
556 rtn_1 = arrayfun(@(x) integral(gt, -0.1, x) / (2*Tb), tn_1);
557 rtn_2 = arrayfun(@(x) integral(gt, -0.1, x) / (2*Tb), tn_2);
558 rtn = arrayfun(@(x) integral(gt, -0.1, x) / (2*Tb), tn);
559
560 % 计算相位
561 rtn_1 = rtn_1 .* pi .* TransferPath(2);
562 rtn_2 = rtn_2 .* pi .* TransferPath(5);
563 rtn = rtn .* pi .* TransferPath(3);
564
565 phase = rtn + rtn_1 + rtn_2 + TransferPath(1);
566
567 % 生成I/Q信号
568 Sn = sqrt(2*E/Tb) * cos(phase);
569 Cn = sqrt(2*E/Tb) * sin(phase);
570 end
571
572 %% GMSK调制函数 - 高分辨率用于显示连续波形
573 function [Sn, Cn, phase, t, gmsk_signal, gmsk_transmit] =
gmsk_modulate_high_res(TransferPath, n, gt, Tb, dt, E, fc)
574 % 高分辨率时间点
575 t = (n*Tb):dt:((n+1)*Tb - dt);
576
577 % 计算高斯脉冲响应
578 rtn_1 = arrayfun(@(x) integral(gt, -0.1, x - (n - 1)*Tb) / (2*Tb), t);
579 rtn_2 = arrayfun(@(x) integral(gt, -0.1, x - (n - 2)*Tb) / (2*Tb), t);
580 rtn = arrayfun(@(x) integral(gt, -0.1, x - n*Tb) / (2*Tb), t);
581
582 % 计算相位
583 rtn_1 = rtn_1 .* pi .* TransferPath(2);
584 rtn_2 = rtn_2 .* pi .* TransferPath(5);
585 rtn = rtn .* pi .* TransferPath(3);
586
587 phase = rtn + rtn_1 + rtn_2 + TransferPath(1);
588
589 % 生成I/Q信号
590 Sn = sqrt(2*E/Tb) * cos(phase);
591 Cn = sqrt(2*E/Tb) * sin(phase);
592
593 % 生成完整的GMSK基带信号（复信号表示）
594 gmsk_signal = Sn + 1i * Cn;
595
596 % 生成发送端GMSK信号（调制到载波）
597 % GMSK信号:  $s(t) = \cos(2\pi f_c t + \phi(t))$ 
598 gmsk_transmit = cos(2*pi*fc*t + phase);
599 end
600
601 %% 维特比解码函数
602 function [path, hamming, hamming_head] = viterbi_decode(Sn, Cn, n, method,
TransferTable, TransferPath, gt)

```

```

603     path = [];
604     temp_path = [];
605     temp_hamming = [];
606     hamming_head = zeros(8,3);
607     hamming = [];
608
609     % 前3个码元的处理
610     if n <= 2
611         index = 1;
612         for i = 1:2^(n+1)
613             if index <= size(TransferTable, 1)
614                 % 生成参考信号
615                 path_segment = [TransferTable(index,(1 + n*3 + 3):(6 +
n*3)), TransferTable(index,(1 + n*3):(1 + n*3 + 2))];
616                 if length(path_segment) >= 6
617                     [Sn1, Cn1] = gmsk_modulate(path_segment, n, gt);
618
619                     % 相关检测
620                     result1 = dot(Sn1, Sn) + dot(Cn1, Cn);
621                     hamming_head(i,n+1) = result1;
622                 end
623                 index = index + max(1, 2^(2 - n));
624             end
625         end
626     else
627         % 后续码元的处理
628         switch method
629             case 'a'
630                 indices = 1:8;
631             case 'b'
632                 indices = 9:16;
633             otherwise
634                 indices = 1:8;
635         end
636
637         for idx = indices
638             if idx <= size(TransferPath, 1)
639                 % 第一条路径
640                 [Sn1, Cn1] = gmsk_modulate(TransferPath(idx,1:6), n, gt);
641
642                 % 第二条路径
643                 path2 = [TransferPath(idx,1:3), TransferPath(idx,7:9)];
644                 if length(path2) >= 6
645                     [Sn2, Cn2] = gmsk_modulate(path2, n, gt);
646
647                     % 相关检测
648                     result1 = dot(Sn1, Sn) + dot(Cn1, Cn);
649                     result2 = dot(Sn2, Sn) + dot(Cn2, Cn);
650
651                     % 选择最佳路径
652                     if result1 >= result2
653                         temp_path = [temp_path; TransferPath(idx, 4:6),
TransferPath(idx,1:3)];
654                         temp_hamming = [temp_hamming; result1];
655                     else

```

```
656         temp_path = [temp_path; TransferPath(idx, 7:9),  
TransferPath(idx,1:3)];  
657         temp_hamming = [temp_hamming; result2];  
658     end  
659 end  
660 end  
661 end  
662  
663     if ~isempty(temp_path)  
664         path = temp_path;  
665         hamming = temp_hamming;  
666     end  
667 end  
668 end
```